Source Code – Smart Campus Booking System

(Campus Resource & Event Management Platform)

Computer Software – Copyright Submission

Introduction

This document presents selected core source code of the Smart Campus Booking System, a full-stack web application developed to simplify campus resource booking and event management within educational institutions. The platform enables students, faculty members, and administrators to efficiently manage campus facilities, organize events, and handle seat-based participation through a centralized digital system.

The application integrates secure user authentication, role-based access control, resource reservation modules, event creation and participation systems, interactive seat selection, and administrative management functionalities. The software is designed using modern MERN Stack technologies including MongoDB, Express.js, React.js, and Node.js to ensure scalability, responsiveness, and efficient performance.

The included source code highlights the primary architecture, backend APIs, database schemas, frontend components, booking workflows, event management logic, authentication mechanisms, and seat allocation functionalities of the application. Only essential modules are provided to demonstrate originality, software structure, implementation methodology, and the core operational features of the system.

==========

BACKEND

==========

File: backend/server.js

```
// Smart Campus Booking System - Backend Server
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./utils/database');
const errorHandler = require('./middleware/errorHandler');

// Load env vars
dotenv.config();

// Connect to database
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/bookings', require('./routes/booking'));
app.use('/api/events', require('./routes/event'));
app.use('/api/users', require('./routes/user'));

// Health check
app.get('/api/health', (req, res) => {
  res.json({ success: true, message: 'Server is running' });
});

// Error handling middleware
app.use(errorHandler);
```

```
const PORT = process.env.PORT || 5000;

const server = app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});

// Handle unhandled promise rejections
process.on('unhandledRejection', (err, promise) => {
  console.log(`Error: ${err.message}`);
  server.close(() => process.exit(1));
});

module.exports = app;
```

File: backend/routes/auth.js

```
const express = require('express');
const { body } = require('express-validator');
const { auth, adminAuth } = require('../middleware/auth');
const { register, login, getMe, createAdmin, initializeFirstAdmin } = require('../
controllers/authController');

const router = express.Router();

// Register validation
const registerValidation = [
  body('name').trim().notEmpty().withMessage('Name is required'),
  body('email').isEmail().withMessage('Please include a valid email'),
  body('password').isLength({ min: 6 }).withMessage('Password must be at least 6
characters'),
  body('role').optional().isIn(['student', 'faculty']).withMessage('Invalid role')
];

// Login validation
const loginValidation = [
  body('email').isEmail().withMessage('Please include a valid email'),
  body('password').notEmpty().withMessage('Password is required')
];
```

```
// Admin creation validation
const createAdminValidation = [
  body('name').trim().notEmpty().withMessage('Name is required'),
  body('email').isEmail().withMessage('Please include a valid email'),
  body('password').isLength({ min: 6 }).withMessage('Password must be at least 6
characters')
];

// Initialize first admin validation
const initializeFirstAdminValidation = [
  body('name').trim().notEmpty().withMessage('Name is required'),
  body('email').isEmail().withMessage('Please include a valid email'),
  body('password').isLength({ min: 6 }).withMessage('Password must be at least 6
characters')
];

router.post('/register', registerValidation, register);
router.post('/login', loginValidation, login);
router.get('/me', auth, getMe);
router.post('/create-admin', auth, adminAuth, createAdminValidation, createAdmin);
router.post('/initialize-admin', initializeFirstAdminValidation, initializeFirstAdmin);
router.post('/temp-create-admin', createAdminValidation, createAdmin);

module.exports = router;
```

File: backend/routes/users.js

```
const express = require('express');
const { body } = require('express-validator');
const { auth, adminAuth } = require('../middleware/auth');
const {
  updateProfile,
  getAllUsers,
  updateUserRole,
  deleteUser
} = require('../controllers/userController');

const router = express.Router();
```

```
// Update profile validation
const updateProfileValidation = [
  body('name').trim().notEmpty().withMessage('Name is required')
];

// Update role validation
const updateRoleValidation = [
  body('role').isIn(['student', 'faculty', 'admin']).withMessage('Invalid role')
];

router.put('/profile', auth, updateProfileValidation, updateProfile);
router.get('/all', auth, adminAuth, getAllUsers);
router.put('/:id/role', auth, adminAuth, updateRoleValidation, updateUserRole);
router.delete('/:id', auth, adminAuth, deleteUser);

module.exports = router;
```

File: backend/routes/booking.js

```
const express = require('express');
const { body, query } = require('express-validator');
const { auth, adminAuth } = require('../middleware/auth');
const {
  createBooking,
  getUserBookings,
  getAllBookings,
  updateBookingStatus,
  deleteBooking,
  getAvailableSeats
} = require('../controllers/bookingController');

const router = express.Router();

// Create booking validation
const createBookingValidation = [
  body('resourceType').isIn(['canteen', 'seminar_hall', 'lecture_hall', 'exploratorium',
'library'])
    .withMessage('Invalid resource type'),
  body('date').isISO8601().withMessage('Valid date is required'),
  body('time').matches(/^([01]?[0-9]|2[0-3]):[0-5][0-9]$/)
```

```
    .withMessage('Time must be in HH:MM format'),
  body('capacity').isInt({ min: 1 }).withMessage('Capacity must be at least 1'),
  body('purpose').trim().notEmpty().withMessage('Purpose is required'),
  body('seats').isArray().withMessage('Seats must be an array'),
  body('seats.*').isString().withMessage('Each seat must be a string')
];

// Update status validation
const updateStatusValidation = [
  body('status').isIn(['pending', 'approved', 'rejected']).withMessage('Invalid status')
];

// Get available seats validation
const getAvailableSeatsValidation = [
  query('resourceType').isIn(['canteen', 'seminar_hall', 'lecture_hall', 'exploratorium',
'library'])
    .withMessage('Invalid resource type'),
  query('date').isISO8601().withMessage('Valid date is required'),
  query('time').matches(/^([01]?[0-9]|2[0-3]):[0-5][0-9]$/)
    .withMessage('Time must be in HH:MM format')
];

router.post('/', auth, createBookingValidation, createBooking);
router.get('/my-bookings', auth, getUserBookings);
router.get('/all', auth, adminAuth, getAllBookings);
router.get('/available-seats', auth, getAvailableSeatsValidation, getAvailableSeats);
router.put('/:id/status', auth, adminAuth, updateStatusValidation,
updateBookingStatus);
router.delete('/:id', auth, deleteBooking);

module.exports = router;
```

File: backend/routes/event.js

```
const express = require('express');
const { body } = require('express-validator');
const { auth, adminAuth } = require('../middleware/auth');
const {
  createEvent,
  findEvent,
  joinEvent,
  leaveEvent,
```

```
  getUserEvents,
  getAllEvents,
  deleteEvent,
  getEventAvailableSeats
} = require('../controllers/eventController');

const router = express.Router();

// Create event validation
const createEventValidation = [
  body('eventName').trim().notEmpty().withMessage('Event name is required'),
  body('description').trim().notEmpty().withMessage('Description is required'),
  body('capacity').isInt({ min: 1 }).withMessage('Capacity must be at least 1'),
  body('dateTime').isISO8601().withMessage('Valid date and time is required')
];

// Find event validation
const findEventValidation = [
  body('eventName').trim().notEmpty().withMessage('Event name is required'),
  body('roomCode').trim().notEmpty().withMessage('Room code is required')
];

// Join event validation
const joinEventValidation = [
  body('eventName').trim().notEmpty().withMessage('Event name is required'),
  body('roomCode').trim().notEmpty().withMessage('Room code is required'),
  body('seats').optional().isArray().withMessage('Seats must be an array')
];

// Leave event validation
const leaveEventValidation = [
  body('eventId').notEmpty().withMessage('Event ID is required')
];

router.post('/', auth, createEventValidation, createEvent);
router.post('/find', auth, findEventValidation, findEvent);
router.post('/join', auth, joinEventValidation, joinEvent);
router.post('/leave', auth, leaveEventValidation, leaveEvent);
router.get('/my-events', auth, getUserEvents);
router.get('/all', auth, adminAuth, getAllEvents);
router.get('/:eventId/seats', auth, getEventAvailableSeats);
router.delete('/:id', auth, deleteEvent);

module.exports = router;
```

==========

MODELS

==========

File: backend/models/User.js

```
// User database model
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,
    trim: true
  },
  password: {
    type: String,
    required: true,
    minlength: 6
  },
  role: {
    type: String,
    enum: ['student', 'faculty', 'admin'],
    default: 'student'
  }
}, {
  timestamps: true
});

module.exports = mongoose.model('User', userSchema);
```

File: backend/models/Event.js

```
const mongoose = require('mongoose');

const eventSchema = new mongoose.Schema({
  eventName: {
    type: String,
    required: true,
    trim: true
  },
  description: {
    type: String,
    required: true,
    trim: true
  },
  organizerId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  roomCode: {
    type: String,
    required: true,
    unique: true,
    uppercase: true
  },
  capacity: {
    type: Number,
    required: true,
    min: 1
  },
  participants: [{
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User'
    },
    seats: [String],
    joinedAt: {
      type: Date,
```

```
      default: Date.now
    }
  }],
  dateTime: {
    type: Date,
    required: true
  },
  totalSeats: {
    type: Number,
    default: 100
  },
  resourceType: {
    type: String,
    default: 'event_hall'
  }
}, {
  timestamps: true
});

module.exports = mongoose.model('Event', eventSchema);
```

File: backend/models/Booking.js

```
const mongoose = require('mongoose');

const bookingSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  resourceType: {
    type: String,
    enum: ['canteen', 'seminar_hall', 'lecture_hall', 'exploratorium', 'library'],
    required: true
  },
  date: {
    type: Date,
    required: true
  },
  time: {
    type: String,
```

```
    required: true
  },
  capacity: {
    type: Number,
    required: true,
    min: 1
  },
  seats: [{
    type: String,
    required: true
  }],
  status: {
    type: String,
    enum: ['pending', 'approved', 'rejected'],
    default: 'pending'
  },
  purpose: {
    type: String,
    required: true,
    trim: true
  }
}, {
  timestamps: true
});

// Index for querying available seats
bookingSchema.index({ resourceType: 1, date: 1, time: 1, status: 1 });

module.exports = mongoose.model('Booking', bookingSchema);
```

==========

MIDDLEWARE

==========

File: backend/errorHandler/auth.js

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const auth = async (req, res, next) => {
  try {
    const token = req.header('Authorization')?.replace('Bearer ', '');

    if (!token) {
      return res.status(401).json({ message: 'Access denied. No token provided.' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id).select('-password');

    if (!user) {
      return res.status(401).json({ message: 'Invalid token.' });
    }

    req.user = user;
    next();
  } catch (error) {
    res.status(401).json({ message: 'Invalid token.' });
  }
};

const adminAuth = (req, res, next) => {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ message: 'Access denied. Admin role required.' });
  }
  next();
};

module.exports = { auth, adminAuth };
```

File: backend/middleware/errorHandler.js

```
const errorHandler = (err, req, res, next) => {
  let error = { ...err };
  error.message = err.message;

  console.error(err);

  // Mongoose bad ObjectId
  if (err.name === 'CastError') {
    const message = 'Resource not found';
    error = { message, statusCode: 404 };
  }

  // Mongoose duplicate key
  if (err.code === 11000) {
    const message = 'Duplicate field value entered';
    error = { message, statusCode: 400 };
  }

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    const message = Object.values(err.errors).map(val => val.message);
    error = { message, statusCode: 400 };
  }

  res.status(error.statusCode || 500).json({
    success: false,
    message: error.message || 'Server Error'
  });
};

module.exports = errorHandler;
```

==========

FRONTEND

==========

File: frontend/pages/Home.tsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

const Home: React.FC = () => {
  const [isVisible, setIsVisible] = useState(false);

  useEffect(() => {
    setIsVisible(true);
  }, []);

  const services = [
    {
      name: 'Canteen',
      description: 'Book tables and arrange meals',
      icon: '🍽️',
      color: 'from-orange-400 to-orange-600',
      bgLight: 'bg-orange-50',
      iconBg: 'bg-orange-100'
    },
    {
      name: 'Seminar Hall',
      description: 'Reserve seminar halls for presentations',
      icon: '🎤',
      color: 'from-purple-400 to-purple-600',
      bgLight: 'bg-purple-50',
      iconBg: 'bg-purple-100'
    },
    {
      name: 'Lecture Hall',
      description: 'Book lecture halls for classes',
      icon: '📚',
      color: 'from-blue-400 to-blue-600',
      bgLight: 'bg-blue-50',
```

```
    iconBg: 'bg-blue-100'
  },
  {
    name: 'Exploratorium',
    description: 'Access innovation labs',
    icon: '🔬',
    color: 'from-green-400 to-green-600',
    bgLight: 'bg-green-50',
    iconBg: 'bg-green-100'
  },
  {
    name: 'Library',
    description: 'Reserve study spaces',
    icon: '📖',
    color: 'from-indigo-400 to-indigo-600',
    bgLight: 'bg-indigo-50',
    iconBg: 'bg-indigo-100'
  },
  {
    name: 'Events',
    description: 'Create and join campus events',
    icon: '🎉',
    color: 'from-pink-400 to-pink-600',
    bgLight: 'bg-pink-50',
    iconBg: 'bg-pink-100'
  }
];

const features = [
  {
    icon: '⚡',
    title: 'Lightning Fast',
    description: 'Book resources in seconds with our intuitive interface',
    color: 'from-blue-400 to-cyan-400'
  },
  {
    icon: '📱',
    title: 'Mobile First',
    description: 'Access the platform from any device, anywhere',
    color: 'from-green-400 to-emerald-400'
  },
  {
```

```
    icon: '🔒',
    title: 'Enterprise Security',
    description: 'Your data is protected with bank-level encryption',
    color: 'from-purple-400 to-pink-400'
  },
  {
    icon: '🎯',
    title: 'Smart Booking',
    description: 'AI-powered recommendations and conflict prevention',
    color: 'from-orange-400 to-red-400'
  },
  {
    icon: '📊',
    title: 'Real-time Analytics',
    description: 'Track usage patterns and optimize resources',
    color: 'from-indigo-400 to-blue-400'
  },
  {
    icon: '🌟',
    title: '24/7 Support',
    description: 'Round-the-clock assistance for all users',
    color: 'from-yellow-400 to-orange-400'
  }
];

return (
  <div className="min-h-screen bg-gradient-to-br from-slate-50 via-blue-50 to-indigo-100 relative overflow-x-hidden">
    {/* Animated Background Elements */}
    <div className="fixed inset-0 overflow-hidden pointer-events-none max-w-screen">
      <div className="absolute -top-40 -right-40 w-80 h-80 bg-purple-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob"></div>
      <div className="absolute -bottom-40 -left-40 w-80 h-80 bg-yellow-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob animation-delay-2000"></div>
      <div className="absolute top-40 left-20 w-72 h-72 bg-pink-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob animation-delay-4000"></div>
    </div>

    {/* Hero Section */}
    <div className="relative py-20 px-4 sm:px-6 lg:px-8">
```

```
      <div className="max-w-7xl mx-auto text-center">
        <div className={`transform transition-all duration-1000 ${isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}>
          <h1 className="text-5xl md:text-7xl font-bold bg-gradient-to-r from-blue-600 via-purple-600 to-indigo-600 bg-clip-text text-transparent mb-6 leading-tight">
            Smart Campus
            <br />
            <span className="text-4xl md:text-6xl">Booking System</span>
          </h1>
          <p className="text-xl md:text-2xl text-gray-600 mb-10 max-w-4xl mx-auto leading-relaxed">
            Transform your campus experience with seamless resource booking and
            <span className="font-semibold text-transparent bg-gradient-to-r from-purple-600 to-pink-600 bg-clip-text"> intelligent event management</span>
          </p>
          <div className="flex flex-col sm:flex-row gap-6 justify-center">
            <Link
              to="/login"
              className="group relative px-10 py-4 bg-gradient-to-r from-blue-600 to-purple-600 text-white font-bold text-lg rounded-xl shadow-lg hover:shadow-2xl transform hover:-translate-y-1 transition-all duration-300 overflow-hidden"
            >
              <span className="relative z-10">Get Started</span>
              <div className="absolute inset-0 bg-gradient-to-r from-purple-600 to-pink-600 opacity-0 group-hover:opacity-100 transition-opacity duration-300"></div>
            </Link>
            <Link
              to="/signup"
              className="group px-10 py-4 bg-white text-blue-600 font-bold text-lg rounded-xl shadow-lg hover:shadow-2xl transform hover:-translate-y-1 transition-all duration-300 border-2 border-blue-100 hover:border-blue-300"
            >
              Create Account
            </Link>
          </div>
        </div>
      </div>
    </div>

    {/* Services Section */}
    <div className="relative py-20 px-4 sm:px-6 lg:px-8">
      <div className="max-w-7xl mx-auto">
        <div className={`text-center mb-16 transform transition-all duration-1000 delay-300 ${isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}>
```

```
          <h2 className="text-4xl font-bold text-gray-900 mb-4">
            Campus Services
          </h2>
          <p className="text-xl text-gray-600 max-w-2xl mx-auto">
            Everything you need, right at your fingertips
          </p>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
          {services.map((service, index) => (
            <div
              key={index}
              className={`group relative bg-white rounded-2xl shadow-lg hover:shadow-2xl transform hover:-translate-y-2 transition-all duration-500 overflow-hidden border border-gray-100 ${isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}
              style={{ transitionDelay: `${600 + index * 100}ms` }}
            >
              <div className={`absolute inset-0 bg-gradient-to-br ${service.color} opacity-0 group-hover:opacity-5 transition-opacity duration-500`}></div>
              <div className="relative p-8">
                <div className={`w-20 h-20 rounded-2xl bg-gradient-to-br ${service.color} flex items-center justify-center mb-6 transform group-hover:scale-110 transition-transform duration-300 shadow-lg`}>
                  <span className="text-3xl filter drop-shadow-md">{service.icon}</span>
                </div>
                <h3 className="text-2xl font-bold text-gray-900 mb-3 group-hover:text-transparent group-hover:bg-gradient-to-r group-hover:bg-clip-text group-hover:from-blue-600 group-hover:to-purple-600 transition-all duration-300">
                  {service.name}
                </h3>
                <p className="text-gray-600 leading-relaxed">
                  {service.description}
                </p>
              </div>
            </div>
          ))}
        </div>
      </div>
    </div>

    {/* Features Section */}
    <div className="relative py-20 px-4 sm:px-6 lg:px-8 bg-gradient-to-br from-gray-50 to-blue-50">
```

```
      <div className="max-w-7xl mx-auto">
        <div className={`text-center mb-16 transform transition-all duration-1000 $
{isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}>
          <h2 className="text-4xl font-bold text-gray-900 mb-4">
            Why Choose Smart Campus?
          </h2>
          <p className="text-xl text-gray-600 max-w-2xl mx-auto">
            Experience the future of campus resource management
          </p>
        </div>

        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
          {features.map((feature, index) => (
            <div
              key={index}
              className={`group bg-white rounded-2xl p-8 shadow-lg
hover:shadow-2xl transform hover:-translate-y-2 transition-all duration-500 border
border-gray-100 ${isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}
`}
              style={{ transitionDelay: `${800 + index * 100}ms` }}
            >
              <div className={`w-16 h-16 rounded-2xl bg-gradient-to-br $
{feature.color} flex items-center justify-center mb-6 transform group-hover:scale-110
transition-transform duration-300 shadow-lg`}>
                <span className="text-2xl">{feature.icon}</span>
              </div>
              <h3 className="text-xl font-bold text-gray-900 mb-3">
                {feature.title}
              </h3>
              <p className="text-gray-600 leading-relaxed">
                {feature.description}
              </p>
            </div>
          ))}
        </div>
      </div>
    </div>

    {/* CTA Section */}
    <div className="relative py-20 px-4 sm:px-6 lg:px-8 bg-gradient-to-br from-
blue-600 via-purple-600 to-indigo-600">
      <div className="relative max-w-4xl mx-auto text-center">
        <div className={`transform transition-all duration-1000 ${isVisible ?
'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}>
          <h2 className="text-4xl font-bold text-white mb-6">
```

```
            Ready to Transform Your Campus Experience?
          </h2>
          <p className="text-xl text-blue-100 mb-10 max-w-2xl mx-auto">
            Join thousands of students and faculty who are already using Smart Campus
          </p>
          <div className="flex flex-col sm:flex-row gap-6 justify-center items-
center">
            <Link
              to="/signup"
              className="group px-10 py-4 bg-white text-blue-600 font-bold text-lg
rounded-xl shadow-2xl transform hover:-translate-y-1 transition-all duration-300"
            >
              Get Started Free
              <span className="block text-sm font-normal text-gray-500 mt-1">No
credit card required</span>
            </Link>
            <div className="flex items-center gap-4 text-white">
              <div className="flex -space-x-2">
                {[1, 2, 3, 4].map((i) => (
                  <div
                    key={i}
                    className="w-10 h-10 rounded-full bg-white/20 border-2 border-
white/30 flex items-center justify-center text-sm font-bold backdrop-blur-sm"
                  >
                    {i}
                  </div>
                ))}
              </div>
              <span className="text-blue-100">k+ Active Users</span>
            </div>
          </div>
        </div>
      </div>
    </div>

  </div>
 );
};

export default Home;
```

File: frontend/pages/Login.tsx

```
import React, { useState, FormEvent, useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

const Login: React.FC = () => {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [isVisible, setIsVisible] = useState(false);
  const { login, loading, error, clearError } = useAuth();
  const navigate = useNavigate();

  useEffect(() => {
    setIsVisible(true);
  }, []);

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();
    clearError();
    await login(email, password);
    if (!error) {
      navigate('/dashboard');
    }
  };

  return (
    <div className="min-h-screen bg-gradient-to-br from-slate-50 via-blue-50 to-indigo-100 flex items-center justify-center py-12 px-4 sm:px-6 lg:px-8 relative overflow-x-hidden">
      {/* Animated Background */}
      <div className="fixed inset-0 overflow-hidden pointer-events-none max-w-screen">
        <div className="absolute -top-40 -right-40 w-80 h-80 bg-purple-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob"></div>
        <div className="absolute -bottom-40 -left-40 w-80 h-80 bg-yellow-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob animation-delay-2000"></div>
        <div className="absolute top-40 left-20 w-72 h-72 bg-pink-300 rounded-full mix-blend-multiply filter blur-xl opacity-20 animate-blob animation-delay-4000"></div>
      </div>
```

```
    <div className="relative w-full max-w-md">
      <div className={`transform transition-all duration-1000 ${isVisible ? 'translate-y-0 opacity-100' : 'translate-y-10 opacity-0'}`}>
        {/* Logo and Header */}
        <div className="text-center mb-8">
          <div className="inline-flex items-center justify-center w-20 h-20 bg-gradient-to-br from-blue-500 to-purple-600 rounded-2xl shadow-2xl mb-4 transform hover:scale-110 transition-transform duration-300">
            <span className="text-white font-bold text-2xl">SC</span>
          </div>
          <h1 className="text-4xl font-bold bg-gradient-to-r from-blue-600 to-purple-600 bg-clip-text text-transparent mb-2">
            Welcome Back
          </h1>
          <p className="text-lg text-gray-600">
            Sign in to access your Smart Campus account
          </p>
        </div>

        {/* Login Form */}
        <div className="bg-white/80 backdrop-blur-lg rounded-2xl shadow-2xl p-8 border border-white/50">
          {error && (
            <div className="mb-6 bg-red-50 border border-red-200 text-red-600 px-4 py-3 rounded-xl flex items-center">
              <span className="text-xl mr-3">⚠️</span>
              {error}
            </div>
          )}

          <form className="space-y-6" onSubmit={handleSubmit}>
            <div className="group">
              <label htmlFor="email" className="block text-sm font-bold text-gray-900 mb-2">
                📧 Email Address
              </label>
              <input
                id="email"
                name="email"
                type="email"
                autoComplete="email"
                required
                value={email}
```

```
              onChange={(e) => setEmail(e.target.value)}
              className="w-full px-4 py-3 border-2 border-gray-200 rounded-xl
focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
transition-all duration-300 hover:border-gray-300 placeholder-gray-400"
              placeholder="Enter your email"
            />
          </div>

          <div className="group">
            <label htmlFor="password" className="block text-sm font-bold text-
gray-900 mb-2">
              🔒 Password
            </label>
            <input
              id="password"
              name="password"
              type="password"
              autoComplete="current-password"
              required
              value={password}
              onChange={(e) => setPassword(e.target.value)}
              className="w-full px-4 py-3 border-2 border-gray-200 rounded-xl
focus:outline-none focus:ring-2 focus:ring-blue-500 focus:border-transparent
transition-all duration-300 hover:border-gray-300 placeholder-gray-400"
              placeholder="Enter your password"
            />
          </div>

          <div className="flex items-center justify-between">
            <div className="flex items-center">
              <input
                id="remember"
                name="remember"
                type="checkbox"
                className="h-4 w-4 text-blue-600 focus:ring-blue-500 border-gray-300
rounded"
              />
              <label htmlFor="remember" className="ml-2 block text-sm text-
gray-900">
                Remember me
              </label>
            </div>
            <div className="text-sm">
              <a href="#" className="font-medium text-blue-600 hover:text-
blue-500">
```

```
              Forgot password?
            </a>
          </div>
        </div>

        <div>
          <button
            type="submit"
            disabled={loading}
            className="group relative w-full flex justify-center py-3 px-4 border border-transparent rounded-xl text-sm font-bold text-white bg-gradient-to-r from-blue-500 to-purple-600 hover:from-blue-600 hover:to-purple-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500 disabled:opacity-50 disabled:cursor-not-allowed transform hover:-translate-y-0.5 transition-all duration-300 shadow-lg hover:shadow-xl overflow-hidden"
          >
            <span className="relative z-10 flex items-center">
              {loading ? (
                <>
                  <div className="w-5 h-5 border-2 border-white border-t-transparent rounded-full animate-spin mr-2"></div>
                  Signing in...
                </>
              ) : (
                <>
                  Sign In
                  <span className="ml-2 transform group-hover:translate-x-1 transition-transform duration-300">→</span>
                </>
              )}
            </span>
            <div className="absolute inset-0 bg-gradient-to-r from-purple-600 to-pink-600 opacity-0 group-hover:opacity-100 transition-opacity duration-300"></div>
          </button>
        </div>
      </form>

      <div className="mt-6">
        <div className="relative">
          <div className="absolute inset-0 flex items-center">
            <div className="w-full border-t border-gray-300" />
          </div>
          <div className="relative flex justify-center text-sm">
            <span className="px-2 bg-white text-gray-500">Demo Accounts</span>
          </div>
```

```
          </div>

          <div className="mt-6 bg-blue-50 border border-blue-200 rounded-xl p-4">
            <p className="font-semibold text-blue-900 mb-3 text-center">Quick
Access Demo Accounts</p>
            <div className="space-y-2 text-sm">
              <div className="flex items-center justify-between p-2 bg-white rounded-
lg">
                <span className="font-medium text-gray-900">Student</span>
                <span className="text-gray-600">student@demo.com / password123</
span>
              </div>
              <div className="flex items-center justify-between p-2 bg-white rounded-
lg">
                <span className="font-medium text-gray-900">Faculty</span>
                <span className="text-gray-600">faculty@demo.com / password123</
span>
              </div>
              <div className="flex items-center justify-between p-2 bg-white rounded-
lg">
                <span className="font-medium text-gray-900">Admin</span>
                <span className="text-gray-600">admin@demo.com / password123</
span>
              </div>
            </div>
          </div>
        </div>

        {/* Sign Up Link */}
        <div className="mt-8 text-center">
          <p className="text-gray-600">
            Don't have an account?{' '}
            <Link
              to="/signup"
              className="font-bold text-blue-600 hover:text-blue-500 transition-colors
duration-300"
            >
              Sign up for free
            </Link>
          </p>
        </div>
      </div>
    </div>
```

```
    </div>
  );
};

export default Login;
```

File: frontend/src/pages/joinEvent.tsx

```
import React, { useState, FormEvent } from 'react';
import axios from 'axios';
import EventSeatMap from '../components/EventSeatMap';

interface JoinForm {
  eventName: string;
  roomCode: string;
  seats: string[];
}

const JoinEvent: React.FC = () => {
  const [formData, setFormData] = useState<JoinForm>({
    eventName: '',
    roomCode: '',
    seats: []
  });
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const [success, setSuccess] = useState('');
  const [joinedEvent, setJoinedEvent] = useState<any>(null);
  const [showSeats, setShowSeats] = useState(false);
  const [seatData, setSeatData] = useState<any>(null);
  const [selectedSeats, setSelectedSeats] = useState<string[]>([]);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: name === 'roomCode' ? value.toUpperCase() : value
    }));
  };
```

```
  const handleSeatsChange = (seats: string[]) => {
    setSelectedSeats(seats);
    setFormData(prev => ({
      ...prev,
      seats
    }));
  };

  const findEvent = async () => {
    if (!formData.eventName || !formData.roomCode) return;

    try {
      const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/
api';
      const response = await axios.post(`${API_URL}/events/find`, {
        eventName: formData.eventName,
        roomCode: formData.roomCode
      });

      if (response.data.event) {
        const event = response.data.event;
        setSeatData(event);
        setShowSeats(true);
      }
    } catch (error: any) {
      setError(error.response?.data?.message || 'Event not found');
    }
  };

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();
    setLoading(true);
    setError('');
    setSuccess('');
    try {
      const API_URL = process.env.REACT_APP_API_URL || 'http://localhost:5000/
api';
      const response = await axios.post(`${API_URL}/events/join`, formData);
      setJoinedEvent(response.data.event);
      setSuccess('🎉 Successfully joined the event!');
      setFormData({
        eventName: '',
        roomCode: '',
        seats: []
      });
```

```
      setShowSeats(false);
      setSelectedSeats([]);
    } catch (error: any) {
      setError(error.response?.data?.message || 'Failed to join event');
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 pt-24 pb-8 relative overflow-x-
hidden">
      <div className="max-w-3xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-gray-900">Join Event</h1>
          <p className="mt-2 text-gray-600">
            Enter the event name and room code to join an existing event
          </p>
        </div>

        {error && (
          <div className="mb-4 bg-red-50 border border-red-200 text-red-600 px-4
py-3 rounded">
            {error}
          </div>
        )}

        {success && (
          <div className="mb-4 bg-green-50 border border-green-200 text-green-600
px-4 py-3 rounded">
            {success}
          </div>
        )}

        {joinedEvent && (
          <div className="mb-6 bg-green-50 border border-green-200 rounded-lg
p-6">
            <h3 className="text-lg font-semibold text-green-900 mb-3">Successfully
Joined Event!</h3>
            <div className="space-y-2">
              <p className="text-green-800">
                <strong>Event Name:</strong> {joinedEvent.eventName}
              </p>
              <p className="text-green-800">
                <strong>Description:</strong> {joinedEvent.description}
```

```
            </p>
            <p className="text-green-800">
              <strong>Date & Time:</strong> {new
Date(joinedEvent.dateTime).toLocaleString()}
            </p>
            <p className="text-green-800">
              <strong>Participants:</strong> {joinedEvent.participants.length}/
{joinedEvent.capacity}
            </p>
          </div>
        </div>
      )}

      <div className="bg-white shadow rounded-lg">
        <form onSubmit={handleSubmit} className="p-6 space-y-6">
          {/* Event Name */}
          <div>
            <label htmlFor="eventName" className="block text-sm font-medium text-
gray-700">
              Event Name
            </label>
            <input
              type="text"
              id="eventName"
              name="eventName"
              value={formData.eventName}
              onChange={handleChange}
              className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-
md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500"
              placeholder="Enter event name"
              required
            />
          </div>

          {/* Room Code */}
          <div>
            <label htmlFor="roomCode" className="block text-sm font-medium text-
gray-700">
              Room Code
            </label>
            <input
              type="text"
              id="roomCode"
              name="roomCode"
              value={formData.roomCode}
```

```
            onChange={handleChange}
            className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-
md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-blue-500"
            placeholder="Enter room code"
            required
           />
          </div>

          {/* Find Event Button */}
          {!showSeats && (
           <button
            type="button"
            onClick={findEvent}
            disabled={loading || !formData.eventName || !formData.roomCode}
            className="w-full flex justify-center py-2 px-4 border border-transparent
rounded-md shadow-sm text-sm font-medium text-white bg-purple-600 hover:bg-
purple-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-purple-500
disabled:opacity-50"
           >
            Find Event & Select Seats
           </button>
          )}

          {/* Seat Selection */}
          {showSeats && (
           <div>
            <h3 className="text-lg font-semibold text-gray-900 mb-4">Select Your
Seats</h3>
            <EventSeatMap
             selectedSeats={selectedSeats}
             onSeatsChange={handleSeatsChange}
             eventData={seatData}
            />
           </div>
          )}

          <div className="flex items-center justify-between">
           <button
            type="submit"
            disabled={loading || !showSeats}
            className="w-full flex justify-center py-2 px-4 border border-transparent
rounded-md shadow-sm text-sm font-medium text-white bg-blue-600 hover:bg-
blue-700 focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-blue-500
disabled:opacity-50"
           >
```

```
              {loading ? 'Joining...' : 'Join Event'}
            </button>
          </div>
        </form>
      </div>

      {/* How to Join */}
      <div className="mt-8 bg-blue-50 border border-blue-200 rounded-lg p-6">
        <h3 className="text-lg font-semibold text-blue-900 mb-3">How to Join an
Event</h3>
        <div className="space-y-3 text-blue-800">
          <div className="flex items-start">
            <span className="font-semibold mr-2">1.</span>
            <p>Get the exact event name and 6-character room code from the event
organizer</p>
          </div>
          <div className="flex items-start">
            <span className="font-semibold mr-2">2.</span>
            <p>Enter the event name exactly as it was created (case-sensitive)</p>
          </div>
          <div className="flex items-start">
            <span className="font-semibold mr-2">3.</span>
            <p>Enter the room code (automatically converted to uppercase)</p>
          </div>
          <div className="flex items-start">
            <span className="font-semibold mr-2">4.</span>
            <p>Click "Join Event" - you'll be added if the event isn't full</p>
          </div>
        </div>
      </div>

      {/* Important Notes */}
      <div className="mt-6 bg-yellow-50 border border-yellow-200 rounded-lg
p-6">
        <h3 className="text-lg font-semibold text-yellow-900 mb-3">Important
Notes</h3>
        <ul className="list-disc list-inside space-y-2 text-yellow-800">
          <li>Both event name and room code must match exactly</li>
          <li>You cannot join an event that's already at full capacity</li>
          <li>You can only join an event once</li>
          <li>Event organizers can see all participants</li>
        </ul>
      </div>
    </div>
  </div>
```

```
  );
};

export default JoinEvent;
```

==========

Components

==========

File: frontend/src/components/EventSeatMap.tsx

```
import React, { useState, useEffect } from 'react';

interface EventSeatMapProps {
  selectedSeats?: string[];
  onSeatsChange: (seats: string[]) => void;
  eventData: any;
}

const EventSeatMap: React.FC<EventSeatMapProps> = ({
  selectedSeats: externalSelectedSeats,
  onSeatsChange,
  eventData
}) => {
  const [selectedSeats, setSelectedSeats] = useState<string[]>(externalSelectedSeats ||
[]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  useEffect(() => {
    if (externalSelectedSeats) {
      setSelectedSeats(externalSelectedSeats);
    }
  }, [externalSelectedSeats]);
```

```
const handleSeatClick = (seat: string) => {
  // Check if seat is already booked by another participant
  const bookedSeats = eventData.participants
    .flatMap((p: any) => p.seats || [])
    .filter((seat: string) => seat && seat.trim() !== ''); // Remove empty seats

  if (bookedSeats.includes(seat)) return;

  setSelectedSeats(prev => {
    let newSeats;
    if (prev.includes(seat)) {
      // Deselect seat
      newSeats = prev.filter(s => s !== seat);
    } else {
      // Select seat
      newSeats = [...prev, seat];
    }
    onSeatsChange(newSeats);
    return newSeats;
  });
};

const generateSeatGrid = () => {
  const rows = 10;
  const cols = 10;
  const bookedSeats = eventData.participants
    .flatMap((p: any) => p.seats || [])
    .filter((seat: string) => seat && seat.trim() !== ''); // Remove empty seats

  const seats = [];
  for (let row = 0; row < rows; row++) {
    for (let col = 0; col < cols; col++) {
      const seatNumber = `${String.fromCharCode(65 + row)}${col + 1}`;
      const isBooked = bookedSeats.includes(seatNumber);
      const isSelected = selectedSeats.includes(seatNumber);

      seats.push(
        <button
          key={seatNumber}
          onClick={() => handleSeatClick(seatNumber)}
          disabled={isBooked}
          className={`
            w-8 h-8 text-xs font-medium rounded transition-all duration-200
            ${isBooked
              ? 'bg-red-500 text-white cursor-not-allowed'
```

```
              : isSelected
                ? 'bg-blue-500 text-white hover:bg-blue-600'
                : 'bg-gray-200 hover:bg-gray-300 text-gray-700'
            }
          `}
        >
          {seatNumber}
        </button>
      );
    }
  }
  return seats;
};

const totalSeats = 100;
const bookedSeats = eventData.participants
  .flatMap((p: any) => p.seats || [])
  .filter((seat: string) => seat && seat.trim() !== ''); // Remove empty seats
const availableSeats = totalSeats - bookedSeats.length;

if (loading) {
  return (
    <div className="text-center py-8">
      <div className="w-8 h-8 border-4 border-blue-200 border-t-blue-600 rounded-
full animate-spin mx-auto mb-4"></div>
      <p className="text-gray-600">Loading seats...</p>
    </div>
  );
}

if (error) {
  return (
    <div className="text-center py-8">
      <div className="text-red-600 mb-4">{error}</div>
      <button
        onClick={() => setError('')}
        className="px-4 py-2 bg-blue-500 text-white rounded-lg hover:bg-blue-600"
      >
        Retry
      </button>
    </div>
  );
}

return (
```

```
    <div className="space-y-4">
      <div className="flex items-center justify-between text-sm">
        <div className="flex items-center space-x-4">
          <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-gray-200 rounded"></div>
            <span className="text-gray-600">Available ({availableSeats})</span>
          </div>
          <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-blue-500 rounded"></div>
            <span className="text-gray-600">Selected ({selectedSeats.length})</span>
          </div>
          <div className="flex items-center space-x-2">
            <div className="w-4 h-4 bg-red-500 rounded"></div>
            <span className="text-gray-600">Booked ({bookedSeats.length})</span>
          </div>
        </div>
      </div>

      <div className="bg-white rounded-lg p-4 border border-gray-200">
        <div className="mb-4 text-center">
          <div className="text-sm text-gray-600 mb-2">Stage / Screen</div>
          <div className="w-full h-2 bg-gray-800 rounded"></div>
        </div>

        <div className="grid grid-cols-10 gap-1 max-w-md mx-auto">
          {generateSeatGrid()}
        </div>
      </div>

      {selectedSeats.length > 0 && (
        <div className="bg-blue-50 border border-blue-200 rounded-lg p-3">
          <p className="text-sm text-blue-800">
            <strong>Selected seats:</strong> {selectedSeats.join(', ')}
          </p>
        </div>
      )}
    </div>
  );
};

export default EventSeatMap;
```

End of Source Code Document

This document contains selected core implementation of the Smart Campus Booking System application.
